Logan Sutton
ID: 11798384
Cpts 360
PA2 Report

All three of my algorithms utilize two functions called begin_process() and add_blocked_process(). The begin_process() function takes a pointer to a process and the opened random file. Then, it determines whether to reset the CPU and IO burst times, and if the CPU remaining time is less than the burst time, the remaining CPU time becomes the CPU burst. The function also resets all list pointers for the process to null, sets the status of the process to 2, and sets the quantum of the process (only used for RR). The add_blocked process() function takes a process and inserts it into the blocked process list from least to greatest remaining CPU time. This ensures that the process that should be unblocked next is the one at the front of the list.

My first-in-first-out algorithm starts by checking whether or not all of the created processes have been completed. Then, the algorithm checks if any blocked processes exist and handles them accordingly. A process that should no longer be blocked is added to the ready queue. All processes also have their time spent blocked attribute incremented, and their IO burst time decremented. Then, after handling the blocked processes, I check whether there are any arrivals. If there are arrivals, I add them to the ready queue. After checking for arrivals, I check if there is currently a process running. If there is, I update the process's current CPU time run and decrease the CPU burst time. I then check whether or not the process is finished. This is part of the tie-breaking criteria. If it is, I update the global total completed processes variable, set the finishing time, and set the current running process pointer to null. If the process has not finished, but its CPU burst time has, I add the process back into the blocked queue and set the current running process pointer to null (again, part of the tie-breaking criteria). Next, I check for if theres not any current running process and if there is a ready process. If this is the case, I start the next ready process. Finally, the last thing I do is update the waiting time of any process that is in the ready queue and then increment the global current cycle variable.

My round-robin algorithm is essentially the same as the FIFO algorithm, except that I keep track of one more queue, the ready-waiting queue. This is the queue where processes are placed when their quantum time runs out. This is checked right after I check for a running process (similar to the FIFO algorithm), and this is checked after the process is considered for being terminated anbd then blocked. The next difference from the FIFO algorithm is that when its time to start a new process, I check the ready queue and ready-waiting queues and apply the tie-breaking criteria. If the waiting-ready queue head process has a lesser arrival time than that of the head process in the ready queue, the waiting-ready queue process is popped and is the new running process. Similarly, the ready queue process would be prioritized if it had a lesser arrival time than the waiting-ready queue. If both arrival times are the same, the tie breaker is then which process was listed earlier in the input. The remainder of the algorithm is essentially the same as FIFO.

My shortest-job-first algorithm is the exact same as the FIFO algorithm, except that when I insert into the ready queue, I insert from the smallest remaining CPU time to the greatest remaining CPU time. This is the only difference, and the tie-breaking criteria are the same as in the FIFO algorithm. The insertion into the ready queue ensures that the shortest job is always run next, which is what should happen by the definition of the algorithm.